CA1 – Navigation and FSM Encounter

Brona Keevers-Roux

Github: https://github.com/hillyleopard133/4th-year-game-dev/tree/main

Video: https://youtu.be/M7S9-xIUMwc

Navigation

Navigation is implemented using custom A* 2D pathfinding based on a grid.

A grid-based approach was chosen over NavMesh to allow explicit control over movement penalties and dynamic cost manipulation, which supports preferred paths and runtime obstacle adaptation.

Navigation Approach

The scene contains a grid and an AStarArea component. The grid maps out the NPC obstacles and preferred paths. The AStarArea component gets a reference to the grid and the upper and lower bounds of the area in which the NPCs will be able to move within.

The NPC target position is converted into grid coordinates, and the A* algorithm computes a sequence of grid nodes of the shortest path from start to goal. A movement penalty is applied to each grid node in the A star area and is taken into account when calculating the route, avoiding obstacles completely and trying to stick to preferred paths.

The computed path is returned as a stack of world positions which are popped sequentially as the NPC moves. The NPCs move using a Rigidbody2D along the path. The NPC automatically updates to the next patrol waypoint or random target wander destination when the path is complete.

Navigation Complication

The A* algorithm can recalculate the movement penalties for the grid nodes at runtime to account for dynamic obstacles and route blockage.

The movement penalties assigned to each grid node take into consideration cost areas/preferred routes.

Performance optimisation

To avoid the constant recalculation of the A* path while chasing the player, the path is only recalculated after the player has moved a set distance from where they were at the time of the previous calculation or after a set amount of time has passed. This prevents unnecessary A* executions every frame while maintaining responsive pursuit behaviour.

FSM

The FSM is implemented using a modular state, action, and decision architecture, allowing behaviour reuse and clear separation of concerns.

States and Transitions

State	Description	Transitions
Patrol	Moves sequentially between predefined waypoints.	Switches to chase when the player is detected in range.
Wander	Moves randomly within a predefined range.	Switches to chase when the player is detected in range.
Chase	Pursues the player.	Switches to attack if player enters attack range. Switches to patrol/wander if player is lost.
Attack	Deals damage to the player at intervals.	Switches to chase if player moves out of attack range.

Interrupt

The detection of the player immediately interrupts the current state to then chase and attack the player.

Robustness

The FSM includes safeguards to maintain stable behaviour during runtime:

- Null target checks are performed before accessing player references, preventing runtime errors when the player is lost.
- The NPC automatically returns to Patrol or Wander when the player exits the detection range.
- Pathfinding is periodically rebuilt during chase behaviour to adapt to player movement and changing navigation conditions.
- Movement coroutines terminate naturally when path steps are exhausted, preventing stalled movement states.

Design trade off

Performance vs Responsiveness

Path recalculation is throttled to reduce computational cost. This introduces minor pursuit delay but significantly improves runtime efficiency.

Grid-Based A* vs NavMesh

A custom grid-based A* solution was implemented instead of using Unity's built-in NavMesh system. While NavMesh provides highly optimised navigation with minimal setup, the grid-based approach offers finer control over movement penalties and dynamic cost adjustment. This allowed preferred paths and runtime cost recalculation to be integrated directly into the pathfinding logic.

However, the grid approach requires manual management of nodes and bounds, and can be less scalable than NavMesh for large environments.

Evaluation

- NPC successfully navigates around static and dynamic obstacles.
- Preferred routes are respected due to movement penalties.
- Chase behaviour remains responsive under throttled path updates.
- System performs reliably under multiple NPC instances.

Overall, the system balances deterministic behaviour control through the FSM with adaptive navigation through A*, resulting in predictable yet responsive enemy encounters.

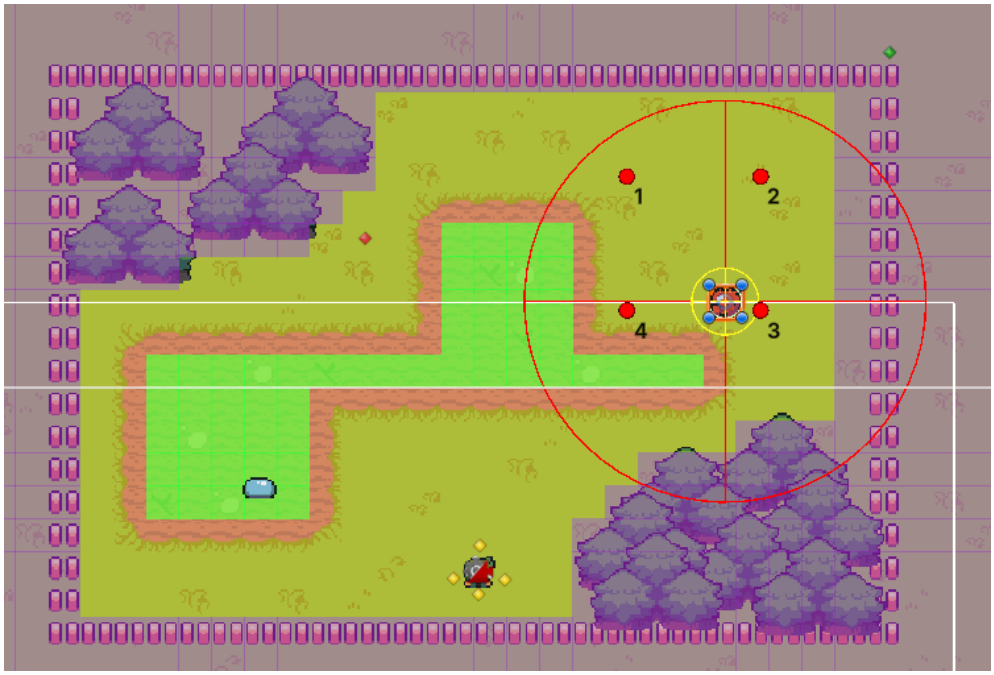
Screenshots

A* obstacles: Purple

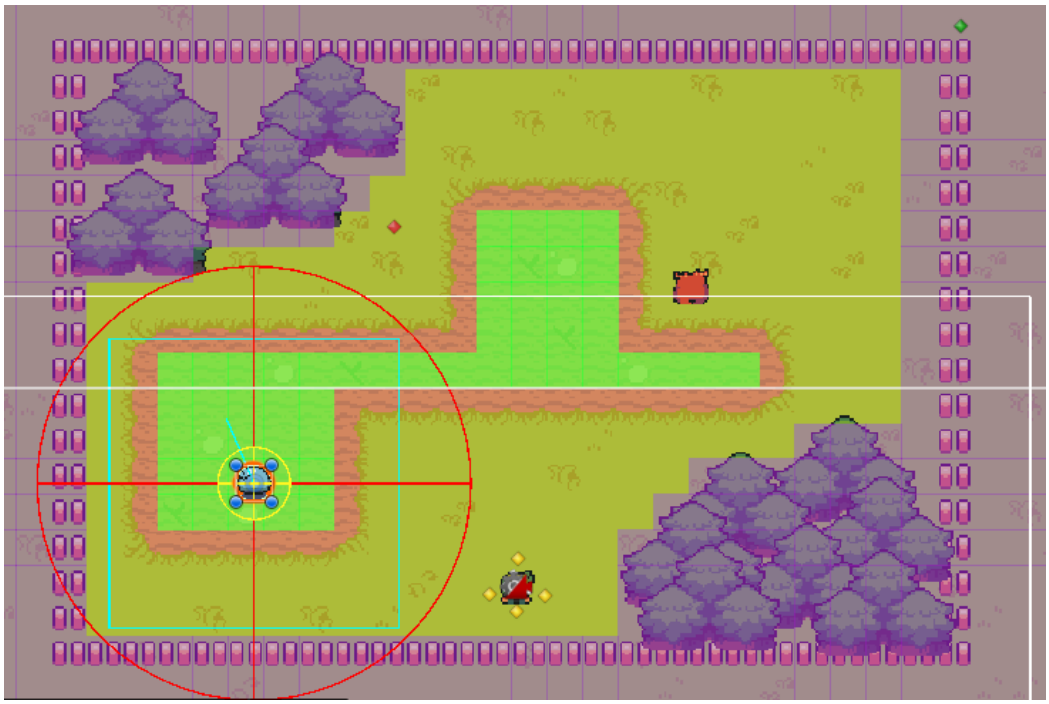
A* preferred paths: Green

Red circle: Player detection range.

Yellow circle: Attack range



An enemy in its patrol state following its numbered waypoints.



An enemy in its wander state, randomly moving towards target destinations within the range of the blue box.

The game view debug overlay displaying the enemy's name, FSM state, current target, target destination position and whether they are alive or not. Blue slime is in attack state as it chased the player and entered within attack range.